Allocation with proof.

A USDC decision engine you can inspect, from source rates to signed consensus.

01

Real mainnet supply APY.

02

Two stages of devnet consensus.

03

Every published allocation has committed inputs.

THE PROBLEM

A yield number is only the beginning.

An allocation also depends on freshness, risk assumptions, concentration limits, and agreement.

01

Markets update on independent clocks. A consumer must reason about the actual age of each input.

02

Independent operators can disagree. Publishing a number does not explain who signed it or why.

03

An apparently valid vote can aggregate into an invalid portfolio. Constraints must survive consensus.

THE PRODUCT

Make the whole decision visible.

One workspace connects live feed data, allocation policy, and the on-chain proof.

01

Observe: read finalized USDC supply rates, feed ticks, source accounts and freshness.

02

Explore: simulate risk discounts and venue caps using the exact consumed input snapshot.

03

Verify: inspect individual consumer vectors, the input hash, decoded instructions and program logs.

Consensus, twice.

Agreement on observations, then agreement on allocation weights.

01

9 producers → 3 median feeds

02

Freeze inputs + policy on chain

03

3 consumers → 1 AllocationState

One feed per producer. Three voices per feed.

Marginfi, Save and Kamino share mainnet USDC, making the comparison concrete.

01

Marginfi: 3 separately signing producers; 120-slot cadence; 240-slot freshness limit.

02

Save: 3 separately signing producers; 180-slot cadence; 360-slot freshness limit.

03

Kamino: 3 separately signing producers; 240-slot cadence; 480-slot freshness limit.

Different clocks. The same consumed inputs.

A RoundContext account fixes the values, ticks, sample slots, eligibility, policy and authorized consumer set.

01

`Published round 509859720`

02

`Snapshot anchor slot 509859730`

03

`SHA-256 10d7df9c596ff35dc177da2f83d5c5ff717bb78680d940dbb8c39331812e98a9`

POLICY

Constraints are part of publication.

The program checks each vote and the final aggregated vector against the frozen policy.

01

50% per-venue cap. Save + Kamino share a 75% illustrative category budget.

02

5% / 10% / 15% illustrative risk discounts. Inputs at or above a 60% discount are screened.

03

Stale or screened positions stay at zero. Infeasible and zero-median results close without overwriting published state.

BUILT AND RUNNING

A complete local devnet fleet.

Separate services make each role and its evidence reviewable.

01

9 | Single-feed producer containers

02

3 | Consumer containers with separate wallets

03

3 | Coordinator, RPC gateway and read-only web services

ENGINEERING

Six confirmed logic issues addressed.

The contract review became reproducible regression checks.

01 Cap-breaking median renormalization → policy-constrained publication or safe abort.

02 Divergence overflow → u128 arithmetic before division.

03 Superseded rounds → terminal, reclaimable accounts.

04 Zero median → terminal no-publication outcome.

05 Window-width changes → immutable feed cadence / new feed required.

06 Contradictory min/max intervals → configuration rejected.

LIVE EVIDENCE

A real published decision.

USDC allocation read from devnet at 2026-10-11T09:40:11.849Z.

01

Marginfi | 50.00%

02

Save | 15.98%

03

Kamino | 34.02%

AllocationState account ↗

Input snapshot ↗

Decoded publication transaction ↗

Demonstrate disagreement and loss.

Five devnet experiments, with recorded transaction signatures.

01 Verified: One extreme producer cannot move the median

02 Verified: Losing one consumer prevents publication

03 Verified: Restart recovers from on-chain round state

04 Verified: Stale inputs do not force an invalid allocation

05 Verified: The full pipeline resumes after feed recovery

VALIDATION

Proof beyond a screenshot.

Unit checks, compiled-program lifecycle checks, devnet transactions, and a browser proof viewer.

01

21 | Native contract tests

02

12 | Local-validator lifecycle checks against the SBF binary

03

45 | RPC/client + allocation JavaScript unit tests

WHAT THIS DEMO PROVES

A decision pipeline with an audit trail.

The end-to-end path is live, bounded and reproducible.

01

Rates from three real mainnet USDC accounts flow through separately signing producer processes.

02

Three consumers agree on frozen asynchronous inputs and publish policy-valid allocation weights on devnet.

03

The presentation can show source provenance, account bytes, transaction decoding, container health and recovery.

WHAT REMAINS

The path from prototype to production.

Each milestone adds a capability or reduces an explicitly known dependency.

01 Expand toward approximately ten distinct feeds across four or five protocols.

02 Integrate the full specified risk methodology and externally grounded category policy.

03 Run independent operators on separate hosts with independent RPC providers.

04 Migrate legacy standalone Rust consumers and harnesses to the policy-bound v2 interface.

05 Evaluate Jito delivery, production monitoring, budget replenishment, and a security audit.

06 Design capital execution, portfolio accounting, and mainnet rollout as separate stages.

NEXT STEP

Inspect a decision. Watch a recovery.

Use the dashboard as the product demonstration, and the proof as the review artifact.

01

1. Open Overview and the consumed-input proof.

02

2. Move a risk or cap slider in the Policy lab.

03

3. Show Fleet status, then the recorded resilience experiments.

Open live demo ↗

Program upgrade transaction ↗